TASK 1 – Understand the Problem and Identify the Data **4 Marks**
Bloom's Level: K4 – Analyse

Analyse the billing problem and identify every item of data the program must store and compute.

Discuss: • The inputs (previous reading, current reading, household number, category) and the outputs (units, energy charge, fixed charge, total)

• Which values never change during a run and should be constants

• Which values can be negative, fractional or very large

• The rules every correct bill must satisfy (for example: a bill is never negative; the total equals the sum of its parts)

• Any assumptions you make

You are expected to justify each data item, not merely list names.

Alternative Concept and Complexity Analysis

If the meter readings were read and stored as text (character strings such as "01530") instead of integers, what would the solution look like? Analyse the time needed to convert and compare each reading (proportional to the number of digits d) and the space used per reading in both representations.

Evidence Expected A clear analysis of the inputs, outputs, constants, rules and assumptions of the billing problem.

Answer :

1. Inputs and outputs

Data item	Type	Justification
Previous meter reading	int	Stores the meter reading recorded during the previous billing period.
Current meter reading	int	Stores the latest meter reading, which must be greater than or equal to the previous reading.
Household number	int or string	Identifies the household. A string is suitable if the number can contain leading zeros.
Category	char or string	Identifies the household's billing category and helps determine the applicable tariff.

Units consumed	int	Calculated by subtracting the previous reading from the current reading.
Energy charge	float or double	Calculated using units consumed and the applicable rate.
Fixed charge	float or double	A fixed amount added according to the billing rules.
Total bill	double	Stores the sum of the energy charge and fixed charge.

Formulas:

- Units consumed = Current reading − Previous reading
- Energy charge = Units consumed × Applicable rate (if a single rate applies)
- Total bill = Energy charge + Fixed charge

The exact energy-charge formula depends on the tariff rules supplied in the assignment.

2. Values that should be constants

Constants are values that should not change during a program run.

Examples:

- Energy rate per unit, if the tariff is fixed.
- Fixed charge, if the tariff specifies a fixed amount.
- Category-specific rates, if they are predefined.
- Any fixed billing limits specified by the problem.

In C, constants can be declared using const, for example:

const double RATE = 5.0;

The value 5.0 is only an example; the actual rate must come from the assignment's billing rules.

Household number, meter readings, units consumed and bill amounts are not constants because they vary between bills.

3. Negative, fractional and very large values

- Negative values: Meter readings, units consumed and bill charges should not be negative under the assumed billing rules. Negative readings must be rejected. If the current reading is below the previous reading, the program should report an error unless meter rollover is explicitly supported.

- Fractional values: Energy charges, fixed charges and the total may contain decimal values. Therefore, double is suitable for monetary calculations, with rounding applied according to the billing rules.
- Very large values: Meter readings and household identifiers may be large. The chosen data type must support the maximum allowed value. long long can be used for large integer readings when necessary.

4. Rules every correct bill must satisfy

1. Both meter readings must be valid and non-negative.
2. The current reading must be greater than or equal to the previous reading, unless meter rollover is supported.
3. Units consumed must equal current reading minus previous reading.
4. Energy charge must follow the specified tariff.
5. Fixed charge must follow the specified billing category.
6. Total bill must equal energy charge plus fixed charge.
7. The final bill must not be negative.
8. Invalid inputs must be detected rather than silently accepted.

5. Assumptions

1. Each household has a unique household number.
2. Meter readings are cumulative readings from the electricity meter.
3. The billing period is represented by the difference between the two readings.
4. Tariff rates and fixed charges are defined by the assignment.
5. The meter has not rolled over unless the problem explicitly states otherwise.
6. Monetary amounts are rounded to the required number of decimal places.

6. Alternative concept: Store meter readings as strings

Instead of storing a reading such as 01530 as an integer, the program could store it as a character string:

char previous Reading[] = "01530";

char current Reading[] = "01620";

Strings preserve leading zeros, which integers do not preserve. However, strings must be validated and converted into numeric values before calculating units consumed.

Time and space complexity

Let \(d\) be the number of digits in one meter reading.

Operation	Integer representation	String representation
Store one reading	O(1) for a fixed-size integer	O(d)space
Convert reading to a number	Not required	O(d) time
Compare two readings	O(1) for fixed-size integers	O(d) in the worst case
Calculate units	O(1)	O(d) if conversion is required first

Explanation: An integer uses a fixed amount of memory for a chosen integer type. A string needs space proportional to the number of digits, plus one character for the terminating '\0' in C. Converting a string to an integer takes time proportional to the number of digits. String comparison can also take \(O(d)\) time in the worst case.

For normal billing calculations, integers are simpler and more efficient when leading zeros do not need to be preserved. Strings are useful when the original text format must be retained.

Conclusion

The billing program must store valid household information and meter readings, calculate units consumed, apply the correct energy and fixed charges, and produce a non-negative total. Selecting suitable data types, defining constants, validating inputs and checking billing rules help ensure accurate results. Storing readings as integers is generally more efficient for calculations, while strings preserve their original formatting.